

RetainIQ

Frontend — Poora Code Explanation

Kese code likha, kyu likha, aur sab kese connect hota hai

Stack: Next.js 14.2.5 + React 18 + TypeScript 5 + TailwindCSS 3 + Recharts + lucide-react

Backend API: FastAPI (<http://localhost:8000>) | Total source files: ~30 (node_modules / .next excluded)

Ye document RetainIQ ke frontend ka har file, har page, har function samjhata hai — simple Hinglish me.

1. RetainIQ Frontend kya hai?

RetainIQ ek customer churn prediction aur retention-ROI platform hai. Iska frontend ek Next.js web app hai jisme bank/fintech ke managers login karke dekhte hain: kaun se customers chhod kar ja sakte hain (churn risk), unko rokne me kitna kharcha aur kitna fayda hoga (ROI), kaun si campaign chalani chahiye, aur A/B test ke results kya rahe.

Frontend ka kaam sirf 3 cheezein hain:

- Backend (FastAPI, port 8000) se data lana aur sundar UI me dikhana — khud koi database ya ML calculation frontend me nahi hota.
- 12 pages ka dashboard shell (sidebar + login guard) dena taaki user ek jagah se sab kuch operate kare.
- Forms, sliders, search/filter, charts ke through user input backend ko bhejna (login, campaign create, ROI simulate, CSV upload).

Tech stack kyu chuna gaya:

Technology	Version	Kyu use kiya
Next.js (App Router)	14.2.5	File-based routing: app/ me folder banao = route ban gaya. Server/client components ka clear split.
React	18.3.1	UI ke liye standard library; hooks (useState/useEffect) se local state + data fetching.
TypeScript	5.5.3	Type safety; API response shape galat ho to build time pe pata chale.
TailwindCSS	3.4.6	Utility classes se bina alag CSS file ke fast styling; poori app me consistent slate/indigo theme.
Recharts	2.12.7	Dashboard ke Pie/Bar/Line charts ke liye; React-friendly.
lucide-react	0.408.0	Sidebar aur logo ke liye clean SVG icons.
fetch (native)	-	Isliye axios nahi: sirf ek chhota wrapper (lib/api.ts) kaafi tha; dependency kam.

Note: Redux/Zustand/SWR jaisi state libraries nahi hain — har page apna data khud useState + useEffect se fetch karta hai. App chhoti hai to ye sabse simple aur samajhne me aasan tareeka hai.

2. Project structure (file tree)

frontend/ folder me kul ~30 asli source/config files hain. node_modules/ (lakho dependency files) aur .next/ (build output) ko hum ignore karte hain — ye commit bhi nahi hote.

Figure 1 — frontend file tree (node_modules aur .next chhod kar)

```

frontend/
+-- package.json, package-lock.json
+-- next.config.js, tsconfig.json, tailwind.config.js, postcss.config.js
+-- next-env.d.ts, .env.example
+-- app/
| +-- globals.css, layout.tsx, page.tsx
| +-- login/page.tsx -> route /login
| +-- dashboard/page.tsx -> route /dashboard
| +-- customers/page.tsx -> route /customers
| +-- customers/[id]/page.tsx -> route /customers/:id
| +-- risk/page.tsx, segments/page.tsx, recommendations/page.tsx
| +-- roi-simulator/page.tsx, campaigns/page.tsx, campaigns/[id]/page.tsx
| +-- experiments/page.tsx, analytics/page.tsx, ai-analyst/page.tsx
| +-- data/page.tsx, settings/page.tsx
+-- components/ui.tsx -> shared design system
+-- lib/api.ts -> backend se baat karne wala client
+-- lib/format.ts -> INR / percent formatting

```

Rule samjho: app/ ke andar jis folder me page.tsx hai, wahi ek route hai. Jaise app/risk/page.tsx ka matlab URL /risk. [id] wale folder dynamic hain — customers/[id]/page.tsx se /customers/5, /customers/42 jaise sab URLs ek hi file se chalte hain.

3. Config files — kese likhe, kyu likhe

3.1 package.json — project ka identity card

Isme 3 cheezein hain: scripts (npm run dev = next dev, build, start), dependencies (next, react, recharts, lucide-react) aur devDependencies (typescript, tailwind, postcss — ye sirf development/build me lagte hain, browser me nahi jate). Deps bahut kam rakhe gaye hain taaki install fast ho aur code samajhna aasan rahe.

3.2 next.config.js — khaali kyu hai?

Ye file lagbhag khaali hai (const nextConfig = {}). Iska matlab default Next.js behaviour kaafi tha — koi custom rewrite, image domain ya env mapping ki zaroorat nahi padi. Backend alag port (8000) par hai aur frontend seedha uska poora URL fetch karta hai, isliye proxy/rewrites ki need nahi.

3.3 tailwind.config.js + postcss.config.js + globals.css

tailwind.config me content: ['./app/**', './components/**'] likha hai — Tailwind sirf in folders me likhi classes ka CSS banata hai, isse final CSS chhota rehta hai. Koi custom color/font nahi — default palette (slate background, indigo primary, red/amber/emerald status) poori app me use hoti hai. postcss.config Tailwind ko Next.js build se jodta hai. globals.css me sirf 3 lines (@tailwind base/components/utilities) + body { bg-slate-100 } — ye ek baar layout.tsx me import hoti hai aur poori app par lagoo hoti hai.

3.4 tsconfig.json — @/* alias kya hai?

paths: { "@/*": ["./*"] } ka matlab: frontend/ folder root se import karo. Isliye har file "@/lib/api" likh sakti hai bajaye "../.../lib/api" ke. strict:true hai (TypeScript sakht checking) lekin code me any kaafi use hua hai taaki API response ke liye zyada type-writing na karni pade.

3.5 .env.example — NEXT_PUBLIC_API_URL

Sirf ek env var hai: NEXT_PUBLIC_API_URL=http://localhost:8000. NEXT_PUBLIC_ prefix zaroori hai — bina iske variable browser tak nahi pahunchta (server tak hi rehta). lib/api.ts isi ko padhkar backend ka address banata hai. Production me isko asli backend URL se badal do, code change ki zaroorat nahi.

4. App ki neev — layout, entry, API client, UI kit

4.1 app/layout.tsx — sidebar shell + auth guard (sabse important file)

"use client" sabse upar likha hai kyu? Kyuki is file me useState/useEffect/localStorage chahiye — ye sab browser me hi chalte hain, server par nahi. File ye karti hai:

- NAV array (12 items: href + label + icon) — sidebar ke saare links ek jagah defined hain, taaki naya page jodna = 1 line jodna.
- useEffect har navigation par: agar path /login hai to kuch mat karo; warna localStorage me retainiq_token nahi mila to router.replace('/login'); mila to GET /api/auth/me se tenant/email lao aur sidebar footer me dikhao.
- Agar /login hai to bina sidebar ke sirf children render karo; warna dark sidebar (w-60 bg-slate-900) + main content (max-w 1400px) ka flex layout do. Active link indigo-600 se highlight hota hai (path === href ya path.startsWith(href)).
- Footer me tenant/email + Sign out button jo logout() chalata hai.

Kyu aise likha: har page me alag-alag auth check likhne ke bajaye ek jagah (layout) me guard lagana DRY principle hai — 14 pages ka code bachta hai aur koi page galti se public nahi rehta.

4.2 app/page.tsx — root route (sirf 2 line)

Listing 1 — root / hamesha /dashboard bhejta hai

```
import { redirect } from "next/navigation";
export default function Home() { redirect("/dashboard"); }
```

Ye server component hai (use client nahi). Koi UI nahi — / kholo to turant /dashboard redirect. Wahan layout guard decide karta hai: token hai to dashboard, nahi to /login.

4.3 lib/api.ts — backend se baat karne ka eklauta darwaza (28 lines)

Listing 2 — lib/api.ts ka poora logic (simplified)

```
const BASE = process.env.NEXT_PUBLIC_API_URL || "http://localhost:8000";
export function token() { return localStorage.getItem("retainiq_token"); }
export function authHeaders() { const t = token(); return t ? { Authorization: `Bearer ${t}` } : {} }
export async function api(path, opts = {}) {
  const res = await fetch(`${BASE}${path}`, { ...opts,
    headers: { "Content-Type": "application/json", ...authHeaders(), ...(opts.headers || {}) } });
  if (res.status === 401 && !path.startsWith("/api/auth")) {
    localStorage.removeItem("retainiq_token"); window.location.href = "/login";
    throw new Error("Session expired");
  }
  const data = await res.json().catch(() => ({}));
  if (!res.ok) throw new Error(data.detail || `Request failed (${res.status})`);
  return data;
}
export function logout() { localStorage.removeItem("retainiq_token"); window.location.href = "/login"; }
```

Kese kaam karta hai, line by line:

- BASE: env var ya default localhost:8000 — har request isi se judta hai, isliye backend ka address badalna = 1 jagah badalna.
- token()/authHeaders(): JWT localStorage key retainiq_token se nikal kar har request me Authorization: Bearer <token> header lagata hai.
- api(path, opts): fetch ka wrapper. JSON header + auth auto-jodta hai. Agar 401 aaya (token galat/expire) aur wo login API nahi thi, to token saaf karke /login bhej deta hai — user ko expire session me atka nahi rehne

deta.

- Error case me backend ka detail message throw karta hai, jo UI me <Err> component dikhata hai. Success me parsed JSON return.
- logout(): token hatao + /login bhejo. Bas.

Do exception yaad rakho: (1) login page raw fetch use karta hai kyuki us waqt token hai hi nahi; (2) data page ka CSV upload FormData bhejta hai (Content-Type JSON nahi lagta, browser khud boundary lagata hai) aur Authorization manually jodta hai.

4.4 lib/format.ts — paise aur percent ek jaisi dikhe

inr(n): number ko Indian grouping me Rs. ke sath dikhata hai (toLocaleString en-IN — lakh/rore style, jaise Rs.12,50,000). pct(n): 0–1 fraction ko percent me (0.87 -> 87.0%). num(n): plain grouped number. Null/NaN par '—' dikhata hai taaki UI crash na ho. Poori app me paise sirf inhi functions se dikhte hain — isliye kahin \$ ya galat format nahi dikhega.

4.5 components/ui.tsx — bina library ka design system (76 lines)

Koi shadcn/MUI nahi — 9 chhote exports jo Tailwind classes lapete hain: Card (white rounded box + title/sub/right header), MetricCard (label + bada value + tone red/green/amber/slate), RiskBadge (High=red, Medium=amber, Low=emerald pill), PrioBadge (Critical/High/Monitor/slate), Loading (pulse), Empty (dashed box), Err (red + Retry), Field (label wrapper), aur 3 class constants inputCls/btnPrimary/btnGhost. Kyu? Har page me same dikhne wale boxes/buttons ke liye copy-paste CSS nahi — ek jagah badlo, 14 pages me badal jaye.

5. Cross-cutting concepts — routing, connection, styling, state

5.1 Routing (Next.js App Router)

Route	File	Kya hai
/	app/page.tsx	Sirf redirect -> /dashboard (server component)
/login	app/login/page.tsx	Public page, sidebar nahi dikhta
/dashboard	app/dashboard/page.tsx	8 KPI + 6 charts (command center)
/customers	app/customers/page.tsx	Search/filter/sort/pagination wali table
/customers/:id	app/customers/[id]/page.tsx	Ek customer ka 360° view (dynamic route)
/risk	app/risk/page.tsx	Model metrics + retrain button
/segments	app/segments/page.tsx	Value x risk quadrants
/recommendations	app/recommendations/page.tsx	Priority wali action queue
/roi-simulator	app/roi-simulator/page.tsx	What-if sliders + live profit calc
/campaigns	app/campaigns/page.tsx	Campaign banao + list
/campaigns/:id	app/campaigns/[id]/page.tsx	Campaign detail + launch + results
/experiments	app/experiments/page.tsx	A/B test ledger (read-only)
/analytics	app/analytics/page.tsx	Product risk + alerts
/ai-analyst	app/ai-analyst/page.tsx	Sawal-jawab chat
/data	app/data/page.tsx	CSV upload/preview + retrain
/settings	app/settings/page.tsx	Workspace + tenant switch + audit

Navigation do tarah se: <Link href> (sidebar, table ke naam — declarative) aur router.replace/push (login ke baad, ROI se campaigns — imperative). Dynamic route me component ko params.id milta hai jisse GET /api/customers/{id} call banta hai.

5.2 Backend connection pattern (har page same formula)

Listing 3 — har page ka standard data-fetch pattern

```
const [data, setData] = useState(null);
const [err, setErr] = useState('');
const load = () => api('/api/dashboard/summary').then(setData).catch(e => setErr(e.message));
useEffect(load, [deps]);
if (err) return <Err msg={err} retry={load} />;
if (!data) return <Loading />;
// ... data se UI render
```

Auth flow: login POST /api/auth/login -> {token} -> localStorage save -> router.replace('/dashboard'). Uske baad har api() call me token jata hai; 401 aaya to auto-logout. Tenant switch (settings) naya token lekar poora page reload karta hai.

5.3 Styling aur charts

Background slate-100, cards white, sidebar slate-900, primary action indigo-600 — ye 4 rang poori app me repeat hote hain. Charts me recharts ke ResponsiveContainer + BarChart/PieChart/LineChart, fixed hex codes (#6366f1 indigo, #ef4444 red...). Icons sirf layout + login me (lucide-react). Koi CSS module ya styled-component nahi — sab inline Tailwind.

5.4 State management — global store kyu nahi?

Har page ka state local useState me hai; server data har mount/filter-change par dobara fetch hota hai (koi cache/SWR nahi). Search input 400ms debounce (customers) aur ROI sliders 300ms debounce (JSON.stringify(f dependency) se backend par load kam rakhte hain. Forms controlled inputs (value + onChange) hain. Token hi akeli global cheez hai (localStorage), tenant/email sirf layout me rehta hai.

6. Har page detail me — kya, kese, kyu, aur backend se kya baat

6.1 Login (/login) — darwaza

Kya: beech me white card (Landmark logo), email+password inputs, Sign in, aur 'ONE-CLICK DEMO LOGIN' me 2 buttons (Demo Bank / Demo Fintech). Kyu demo buttons: evaluator ko typing ke bina turant andar jane ke liye. Kese: submit() me POST /api/auth/login {email, password:'demo123'} — dhyan do, typed password ignore karke demo123 bheja jata hai (demo convenience). Jawab me token -> localStorage -> /dashboard. API: sirf POST /api/auth/login.

6.2 Dashboard (/dashboard) — executive command center

Kya: 8 MetricCards (total customers, high-risk count, revenue at risk = prob×CLV ka jod, expected protected, retention budget, campaign cost, expected ROI %, targeted/retained) + 6 visuals: risk Pie, segment Bar (tooltip me inr), risk-trend Line, product bars (max se % width), quadrant 2x2 grid (median CLV par split), campaign table. Kese: load() me Promise.all se 7 GET ek sath (/dashboard/summary, risk-dist, revenue-by-segment, product-risk, risk-trend, quadrant, campaign-perf). Kyu ek sath: 7 round-trip ki waiting ek me nipat jati hai. Backend connection: saare /api/dashboard/* endpoints.

6.3 Customers (/customers) — ranked ledger

Kya: filter grid (search input + risk/segment/sort selects + count) + table (Customer link -> RiskBadge -> proba pct -> CLV inr -> RaR inr -> action -> PrioBadge) + Prev/Next pagination (page_size=20). Kese: GET /api/customers?search&risk;&segment;&sort;&page;&page;_size=20. Search 400ms debounce + page reset; baaki filter turant. Kyu RaR sort default: sabse zyada paisa-dao-par customer upar dikhe — manager ka time sabse valuable jagah lage.

6.4 Customer detail (/customers/:id) — 360° explainability

Kya: header (naam + demographics + Risk/Prio badges), 4 tiles (proba, CLV, RaR, deterioration/100), 'Why at risk?' card (explanation sentence + drivers list: red dot = risk badhata, green = protective; value vs typical + upar/neeche arrow), trajectory LineChart (d90->d60->d30->today se bigadta trend), Next Best Action card (kaun si action, kyu, cost/success/ROI grid), Profile card (products, balances, txn/login/complaint counts, annual contrib, recent txns scroll). Kese: params.id se GET /api/customers/{id}; traj array derive karke chart data. Kyu itna detail: manager ko sirf score nahi, wajah + ilaaj dono chahiye — warna action nahi lega.

6.5 Risk (/risk) — model transparency + retrain

Kya: header + Retrain button, risk BarChart, Model eval card (trained hai to AUC/Precision/Recall/F1 3-decimal + TN/FP/FN/TP 2x2; nahi to note), 'How prediction works' card (17 features, bands: Low<35%, Med 35–60%, High ≥60%). Kese: Promise.all([risk-dist, predictions/metrics]); retrain() POST /api/predictions/retrain -> msg + reload. Kyu: model black-box na lage — accuracy numbers + retrain control manager ka bharosa banata hai.

6.6 Segments (/segments) — value × risk quadrants

Kya: 2-col grid cards — har segment ka naam, right me uska RaR, count + avg proba, aur progress bar (sabse bade RaR se normalize). Kese: GET /api/segments/overview -> {segments[], median_clv}. Kyu: poora paisa kahan fansa hai, ek nazar me.

6.7 Recommendations (/recommendations) — action queue

Kya: priority select + table (Customer + risk/RaR sub-line -> PrioBadge -> Action -> Why -> Cost -> Success -> ROI green bold). Kese: GET /api/recommendations?priority=&limit;=100, filter change par reload. Kyu expected-net order me sorted: pehle wo customer jis se sabse zyada bachne layak paisa hai.

6.8 ROI Simulator (/roi-simulator) — hero feature

Kya: left Inputs card (5 range sliders: min_proba, min_clv, intervention_cost, success_rate, reach — live labels + audience/break-even line), right 6 MetricCards + Sensitivity BarChart (conservative/expected/optimistic 3 rang) + 3 scenario tiles + CTA button -> /campaigns. Kese: sliders badalte hi 300ms debounce me POST /api/roi/simulate; jawab {targeted, retained, cost, revenue, net, roi_pct, scenarios, audience, avg_clv, break_even_rate}. Formula UI me likha hai: retained = targeted × success, revenue = retained × avg CLV. Kyu sliders: manager 'agar success 28% ke bajaye 35% ho to?' turant dekh sake — decision gate yahi hai.

6.9 Campaigns (/campaigns) — campaign studio

Kya: 2-col — left New campaign card (name, 7 interventions ka select: fee_waiver/cashback/personalized_offer/loan_offer/premium_upgrade/rm_call/service_recovery, 5 numeric fields, Create), right All campaigns table (name, intervention, status pill, targets, Open link). Kese: GET list + POST /api/campaigns -> msg 'Projected: audience, revenue, ROI' + reload. Kyu draft-then-launch: pehle projection dekho, pasand aaye tabhi launch.

6.10 Campaign detail (/campaigns/:id) — execution + results

Kya: header + Launch button (completed nahi hai to), phir draft card (80/20 split note) YA do cards: Control vs Treatment (retention tiles + lift) + Financial outcome (cost/revenue/ROI). Kese: GET /api/campaigns/:id; launch() POST /api/campaigns/:id/launch. Kyu 80/20: treatment (offer wale) vs control (bina offer) — lift sabit karta hai campaign ne hi bachaya.

6.11 Experiments (/experiments) — A/B ledger

Kya: read-only table (Experiment+campaign -> treat/ctrl n -> treat ret -> ctrl ret -> lift green -> revenue -> ROI; bina result 'pending launch'). Kese: GET /api/experiments. Kyu read-only: experiment result me haath se edit ka koi matlab nahi — ye history hai.

6.12 Analytics (/analytics) — product risk + alerts

Kya: upar Alerts card (top 8 red banners, customer link sath), neeche 2-col: Product table (product, customers, high-risk red, RaR) + Deterioration scroll list (top 25, flags + amber score). Kese: Promise.all 3 GET (/analytics/products, deterioration, alerts). Kyu: alerts se seedha customer detail link — dekho -> click -> action.

6.13 AI Analyst (/ai-analyst) — sawal-jawab chat

Kya: max-w-3xl chat — suggestion pills, input + Ask (Enter bhi chalta hai), messages: right indigo bubble (tumhara sawal, ml-8) + left white card (jawab, mr-8, whitespace-pre-wrap). Kese: mount par GET /api/ai/suggested -> pills; ask() POST /api/ai/ask {question} -> log me {q,a} prepend, input saaf; error to 'Error: msg'. Kyu grounded: jawab hamesha asli DB numbers se banta hai (backend context), hallucination nahi.

6.14 Data (/data) — CSV ingest + retrain

Kya: max-w-2xl — status header, Upload card (Choose CSV label, Download sample link \${BASE}/api/data/sample, Retrain button), Preview card (detected type, rows stats, telco_note banner, mapping table 8 rows, issues, Import button). Kese: postFile() helper FormData('f') + manual Bearer (JSON header nahi!) se POST /api/data/preview phir /api/data/upload -> msg 'added + type + rejected + errors' + reload; retrain alag POST. Kyu preview-then-import: galat file turant pakdi jaye, ganda data DB me na ghuse.

6.15 Settings (/settings) — workspace + audit

Kya: Workspace card (user/tenant, synthetic-data disclaimer, tenant switch input+button), Audit card (scroll max-h-96: ts/action/meta rows). Kese: Promise.all([/api/auth/me, /api/audit?limit=50]); sw() POST /api/auth/switch?tenant_id= -> naya token -> window.location='/dashboard' (poora reload taaki sab data naye tenant ka aaye). Kyu: multi-tenant demo (Demo Bank vs Demo Fintech) ek click me switch.

7. End-to-end user journey (poora flow ek najar me)

Figure 2 — login se retention tak ka data flow

```
1. LOGIN (/login): POST /api/auth/login -> token -> localStorage -> /dashboard
2. SHELL (layout, har page par): token nahi -> /login | token hai -> GET /api/auth/me (tenant/email)
3. DASHBOARD: 7x GET /api/dashboard/* -> KPI + charts -> pata chala 'High-Value-High-Risk' me paisa fansa hai
4. CUSTOMERS: GET /api/customers?... -> RaR table -> naam click -> /customers/:id
  DETAIL: GET /api/customers/:id -> wajah (drivers) + ilaaj (next best action)
5. RISK/SEGMENTS/RECOMMENDATIONS: metrics + quadrants + priority queue
6. ROI SIMULATOR (decision gate): sliders -> POST /api/roi/simulate (300ms) -> profit dikha -> CTA /campaigns
7. CAMPAIGNS: POST /api/campaigns (projection) -> GET list -> GET /:id -> POST /:id/launch (80/20 test)
  EXPERIMENTS: GET /api/experiments -> lift/revenue/ROI history (+ dashboard campaign-perf)
8. ANALYTICS + AI ANALYST: products/deterioration/alerts | suggested + POST /api/ai/ask chat
9. DATA (loop-back): CSV preview -> upload (added/rejected) -> POST /predictions/retrain -> dashboard refresh
10. SETTINGS: /api/auth/me + /api/audit | tenant switch -> naya token -> reload
  Har request me Authorization: Bearer token; 401 par auto /login. Paise inr() se, percent pct() se.
```

7.1 Frontend kaise chalaye

Listing 4 — frontend start commands

```
cd frontend
npm install
# .env.local me NEXT_PUBLIC_API_URL=http://localhost:8000 (ya .env.example copy karo)
npm run dev # http://localhost:3000
# Demo login: admin@demobank.in / demo123 (backend seed ke baad)
```

Yaad rakho: pehle backend (port 8000) aur seed data chahiye, warna dashboard khaali dikhega — frontend me koi apna data nahi rehta.